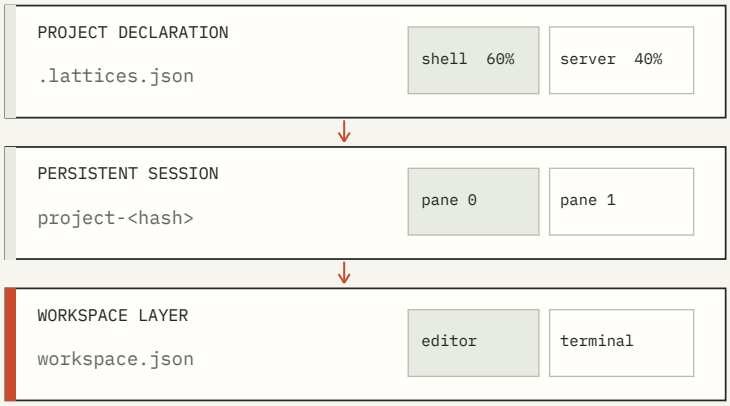
SUBSYSTEM 01 / ORGANIZE

Workspace system

Keep a project's working context in named sessions, tab groups and layers. Inspect the desktop without losing the distinction between saved and live state.

FIG. B / CONTEXT ASSEMBLY

SCHEMATIC / NOT TO SCALE



01 Declare

Panes, commands, placement

02 Resume

Attach, sync, restart

03 Compose

Match, focus, tile, reveal

CONFIGURATION -> SESSION STATE -> DESKTOP MEMBERSHIP

FUNCTIONAL SPECIFICATION

01 Projects & development

Discover configs, choose among six terminals and launch sessions. Detect dev/build commands for Node, Swift, Rust, Go and Make.

02 Session lifecycle

Declare named panes and sizes. List, detach or kill sessions; sync missing panes; restart commands; ensure or prefill on reattach.

03 Tab groups

Launch configured project groups and cycle tabs. Build temporary stacks from live windows, then dissolve them when finished.

04 Layer rules

Match projects, apps or specific windows. Create, rename, delete, capture and edit layers; add or remove membership.

05 Layer switching

Activate, launch and arrange a layer. Park or hide other windows; Show All restores them. Eight direct slots surround a center status key.

06 Workspace views

Overview, Studio and HUD maps expose windows and layers. Inspect saved, last-known and live positions; preview, focus and arrange.

07 Find context

Search titles, apps, session tags and OCR. Deep search adds live terminal titles, working directories, commands and tmux context.

08 Maps & inventory

Read the desktop as an ASCII map or JSON. Inspect managed sessions, terminal instances, processes, display frames and window IDs.

STATE CONTRACT

Project config defines panes. workspace.json defines layers and groups. Live membership resolves against current windows; CLI map reads do not move them.

DRAWING
LAT-FS-01

ISSUE / BASIS
A / Working source

SOURCE SNAPSHOT
02 Oct 2026

SHEET / SERIES
02 / 04